UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época Normal • 16 de janeiro de 2023 • 16:00 – 18:00

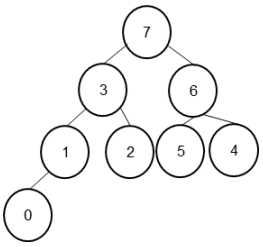
- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas lápis/caneta, identificação, este enunciado, e folha de teste.

Grupo I: Perguntas de escolha múltipla

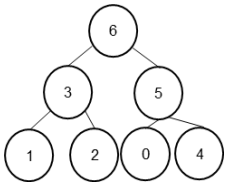
1. (0.5) Considere uma árvore de pesquisa binária de inteiros que se encontra inicialmente vazia. Seguidamente são inseridos nesta árvore, sequencialmente e por ordem crescente, todos os números inteiros entre 1 e n. O número médio de operações necessárias para procurar por um número que não faz parte da árvore é:

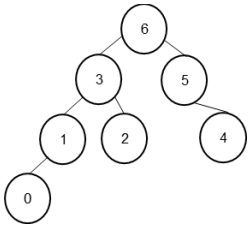
a) n.
b) $\log_2 n$
c) n/2
d) $\frac{\log_2 n}{2}$

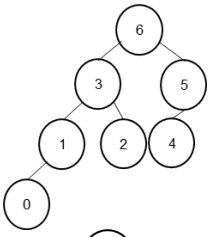
2. (0.5) Considere o seguinte *heap*:

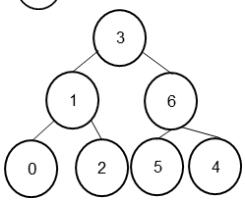


Qual dos seguintes *heaps* corresponde ao resultado de se remover o elemento 7 do heap?

a)


b)


c)


d)


3. (0.5) Considere o método *partition* do algoritmo de ordenação *quicksort* aplicado ao seguintes argumentos: a = [4, 2, 5, 0, 1, 7, 3, 8], low=0, high = 7.
Indique qual dos seguintes arrays é o resultado da invocação ao método *partition*, sabendo que o pivô é escolhido como sendo o elemento mais à esquerda.

a)
[0, 1, 2, 3, 4, 5, 7, 8]

b)
[1, 2, 3, 0, 4, 7, 5, 8]

c)
[4, 2, 3, 0, 1, 7, 5, 8]

d)
[7, 2, 3, 0, 1, 4, 5, 8]

4. (0.5) Para um determinado algoritmo, que sabe ter complexidade assimptótica de O($\log_2$ n), indique qual seriam os resultados teóricos esperados para o valor de razão dobrada r, caso fizesse ensaios de razão dobrada:

a) r = 4
b) r = 8
c) r começa em 4 e vai baixando até r=2.
d) r começa em 3 e vai baixando até r=1

5. (0.5) Indique a aproximação tilde mais correta para a seguinte expressão:

$$2n^2 + \frac{5n^3}{n} + n^3 + 5n + \log_2 n$$

Selecione uma opção de resposta:
a) $\sim 7n^2$
b) $\sim 5n^2$
c) $\sim 6n^3$.
d) $\sim n^3$

6. (0.5) Considere as seguintes afirmações:
1. Na implementação de uma tabela de dispersão, a escolha de uma boa função de dispersão assegura sempre a inexistência de colisões.
2. Numa tabela de dispersão com dupla dispersão temos de usar remoção preguiçosa quando queremos apagar uma chave.
3. Numa tabela de dispersão com exploração linear usam-se duas funções de dispersão.
4. Uma operação de inserção de uma chave que já existe na tabela corresponde a uma operação de atualização.

Quais das afirmações acima são verdadeiras:
a) nenhuma
b) 1, 2 e 4
c) todas
d) 2, 3 e 4
e) 1, 2
f) 2, 4

Grupo II: Perguntas de desenvolvimento

7. (2.0) Exercícios sobre a linguagem de programação Java.
a) (1.0) Escreva um método estático chamado fatorial, que recebe um inteiro n, e calcula o fatorial de n (lembre-se que o fatorial de n é dado pela multiplicação de todos os elementos de 1 até n (inclusive)).

b) (1.0) Usando o método implementado na alínea anterior, implemente um método estático, designado de *calculaFatoriais* que recebe um número n (sendo n um inteiro positivo), e que devolve um array de inteiros de tamanho n, em que o array contém todos os fatoriais de 1 até n. Por exemplo calculaFatoriais(5) devolverá o array [1,2,6,24,120].

8. (2.0) Resolva os seguintes exercícios sobre análise de complexidade temporal.
a) (1.0) A Maria pretendia implementar uma função em que dada uma coleção de nomes (que poderá ter vários nomes repetidos), é retornada uma coleção apenas com os nomes únicos não repetidos, ordenados por ordem alfabética. A sua implementação foi a seguinte:

```
public static Collection<String> getSortedCollectionOfUniqueNames(Collection<String>
names)
{
    List<String> uniqueNames = new ArrayList<String>();

    for(String s : names)
    {
```

```
        if(!uniqueNames.contains(s)){uniqueNames.add(s);}
    }

    //aqui será invocado o quicksort com o comparador por omissão de strings
    uniqueNames.sort(null);
    return uniqueNames;
}
```

Indique a complexidade temporal assimptótica para o método *getSortedCollectionOfUniqueNames* em função do tamanho *n* da coleção recebida como argumento.

b) (1.0) Embora cumpra a funcionalidade pretendida, o método implementado pela Maria não é muito eficiente. Explique como é que poderia implementar a mesma funcionalidade de forma substancialmente mais eficiente usando outra estrutura de dados (não necessita de implementar). Qual seria a complexidade temporal assimptótica para a nova versão proposta?

9. (2.0) Considere uma pilha (*stack*) e uma fila (*queue*) inicialmente vazias, sobre as quais é executada a seguinte sequência de instruções:

```
StackList<Integer> s = new StackList<Integer>();
QueueList<Integer> q = new QueueList<Integer>();
int j;

s.push(4)
s.push(7);
q.enqueue(2);
q.enqueue(1);

for(int i = 0; i<3; i++)
{
    s.push(q.dequeue());
    q.enqueue(i);
    s.push(q.dequeue());
    q.enqueue(s.pop());
    q.enqueue(s.pop());
}
while(!s.isEmpty()) System.out.println(s.pop() + "/");
```

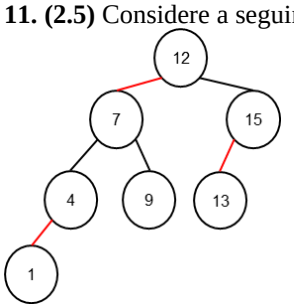
Represente a fila q e a pilha s, com os elementos introduzidos e removidos ao longo da execução do código e indique a sequência impressa no ecrã quando o código é executado.

10. (2.5) Suponha que o algoritmo de ordenação bubblesort *é* invocado com o seguinte array:

a=[5, 7, 3, 8, 1, 6, 4, 9, 2,10]

Indique os vários estados do array para as primeiras 3 iterações principais do algoritmo (que correspondem ao ciclo exterior do algoritmo).

11. (2.5) Considere a seguinte árvore rubro-negra (red-black):



a) (1.5) Insira a chave 5 na árvore, desenhando os principais passos intermédios, e indique explicitamente quando efetuar uma das seguintes operações: Rotate Left, Rotate Right, e Flip. Desenhe ligações vermelhas a tracejado e ligações negras com linhas contínuas. Desenhe também a árvore final.

b) (1.0) Insira a chave 17 **na árvore original**, desenhando os principais passos intermédios, e indique explicitamente quando efetuar uma das seguintes operações: Rotate Left, Rotate Right, e Flip. Desenhe ligações vermelhas a tracejado e ligações negras com linhas contínuas. Desenhe também a árvore final.

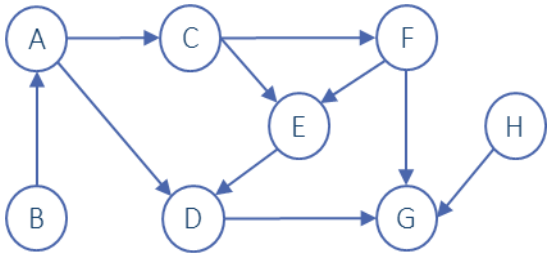
12. (1.0) Considere que necessita de implementar uma estrutura de dados, onde existem 3 tipos de operações que vão ser muito frequentes: inserir um novo elemento; obter e remover o elemento mais pequeno; obter e remover o maior elemento. Nesta situação, entre um amontoado (*heap*) e uma árvore binária de pesquisa, qual seria a escolha mais apropriada para implementar a sua estrutura de dados? Justifique.

13. (2.5) Considere uma tabela de dispersão com técnica de cuco, como implementada no projeto 4. Considere também que são usadas como funções de dispersão, as funções $h_0(k)$ e $h_1(k)$ que determinam a posição da chave na tabela t_0 e na tabela t_1, dadas pelas seguintes expressões:
$h_0(k) = \big(abs(hashcode(k)\big)\ \%\ m_0$
$h_1(k) = \big(abs(hashcode(k)\big)\ \%\ m_1$
Onde m_0 corresponde ao tamanho da tabela t_0, e m_1 ao tamanho da tabela t_1.*abs* corresponde a uma função de retorna o valor absoluto de um número.

a) (1.5) Considere o caso em que foi inicializada uma nova tabela de dispersão de cuco cujas chaves são inteiros (em java, o *hashcode* de um inteiro é o próprio inteiro), e com tamanho inicial de $m_0 = 5$, e $m_1 = 7$. Indique o estado das tabelas de dispersão t0 e t1, à medida que forem inseridas as 5 chaves, pela seguinte ordem: 10, 5, 1, 6, -1.

b) (1.0) A seguir à inserção das 5 chaves anteriores, foi inserida a chave 36. Explique o acontece quando esta chave é inserida.

14. (2.5) Considere o grafo dirigido da figura seguinte:



a) (1.5) Faça uma procura em profundidade primeiro sobre o grafo, com origem no vértice **B**. Considere que os vértices adjacentes de um vértice são visitados por ordem alfabética. Indique a ordem e a pós-ordem pela qual os vértices são visitados pela procura.

b) (1.0) Caso exista, indique uma sequência de vértices que corresponda a uma ordenação topológica do grafo. Caso não exista, justifique porquê.